This is a valid concern without an easy answer. I tend to favor simple business-logic checks inside the domain, without relying on the framework's authorization infrastructure, to make my domain easier to test and framework-independent. But doing so loses some of the benefits of such a framework:

- The `IAuthorizationService` uses declarative policies, even though you are calling the authorization framework imperatively.
- You can decouple the need to authorize an action from the actual requirements.
- You can easily rely on peripheral services and properties of the request, which may be harder (or undesirable) with business logic checks.

You can achieve these benefits in business-logic checks, but that typically requires creating a lot of infrastructure too, so you lose a lot of the benefits of keeping things simple. Which approach is best will depend on the specifics of your application design, and there may well be cases for using both.

For example, one possible approach is to use the basic `[Authorize]` attribute as described in section 24.2.1 to prevent anonymous access to your APIs, potentially with simple, coarse policies applied to your APIs. You would then rely on "manual" business-logic checks against the `ClaimsPrincipal` in your domain as required. This may reduce a lot of the complexity and indirection associated with the ASP.NET Core authorization system.

24.6 Hiding HTML elements from unauthorized users

All the authorization code you've seen so far has revolved around protecting endpoints on the server side, rather than modifying the UI for users. This is important and should be the starting point whenever you add authorization to an app.

WARNING Malicious users can easily circumvent your UI, so it's important to always authorize your endpoints on the server, never on the client alone.